



Sed and Awk

Amitabha Sanyal
April - 2026



The sed model

sed is a *stream editor*

usage *sed options script input_files*

sed cycle:

Until all lines are read:

read the next line from the input file into a buffer called pattern space

- a. process the line in the pattern space according to the script
- b. transfer the contents of pattern space to the output stream.

A separate buffer called hold space is used by some sed commands to hold and accumulate text between cycles.

The script may be

- a. inlined: `sed 'inlined_script' input_files`, or
- b. read from a file: `sed -f script_file_name input_files`

[This](#) is the main link for sed.



Common Options

These are:

-n : Don't print the pattern space by default. So output appears only when we explicitly request it.

```
sed -n '/Deceased/p' covid_data.csv
```

-f scriptfile: Pick the script from **scriptfile**

-s : applied command on each file separately

```
sed -n -s '1,10 p' Students.csv covid_data.csv
```

otherwise the command is applied to the concatenation of files

-r : Enables usage of extended regular expressions

[This link](#) gives the complete list of options in sed.



Script structure

A script is of the form:

[*address*] [!] *command*

. . .

[*address*] [!] *command*

A missing address means all lines. ! is negation: means all lines except the lines specified by *address*.

address can be

- A single-line address

- A set-of-lines address

- Range address

- Nested address



Address

single line address.

show only line 3

```
sed -n '3 p' Students.csv
```

show only last line

```
sed -n '$ p' Students.csv
```

substitute "HW" with "ML" on line 10 and print the line

```
sed -n -e '10 s/HW/ML/' -e '10 p' Students.csv
```

note use of -e to join more than one commands in the command line.

set-of-lines address using regular expressions

Show all dual degree students

```
sed -n -r '/19D[0-9]{6}/ p' Students.csv
```

note use of extended regular expressions using -r

Show all directories in the current directory

```
ls -al | sed -n '/^d/ p'
```



Address (contd.)

range address.

show the lines between 1 and 10

```
sed -n '1,10 p' spy.py
```

- show the lines between an if statement and line 10:

```
sed -n '/^if/,10 p' spy.py
```

- show the lines between a `if` and *its closest* `else`:

```
sed -n '/^if/,/^else/ p' spy.py
```

nested address

Show all `print` statements between 20 and 30

```
sed -n '20,30{/print/ p}' spy.py
```

complement address (!)

Show all statements other than `print` between 20 and 30

```
sed -n '20,30{/print/! p}' spy.py
```

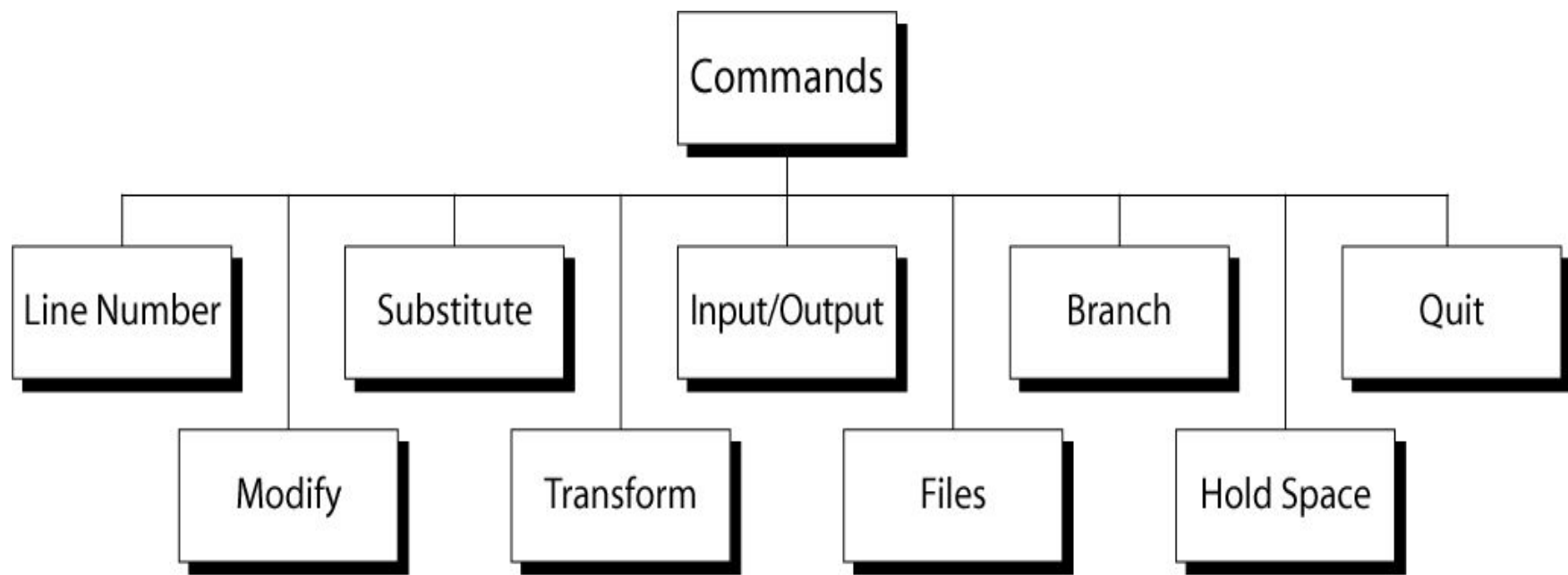
Proprietary content. ©Copyright protected. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by cgodhandaraman@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.



Categories of sed commands address





Line command

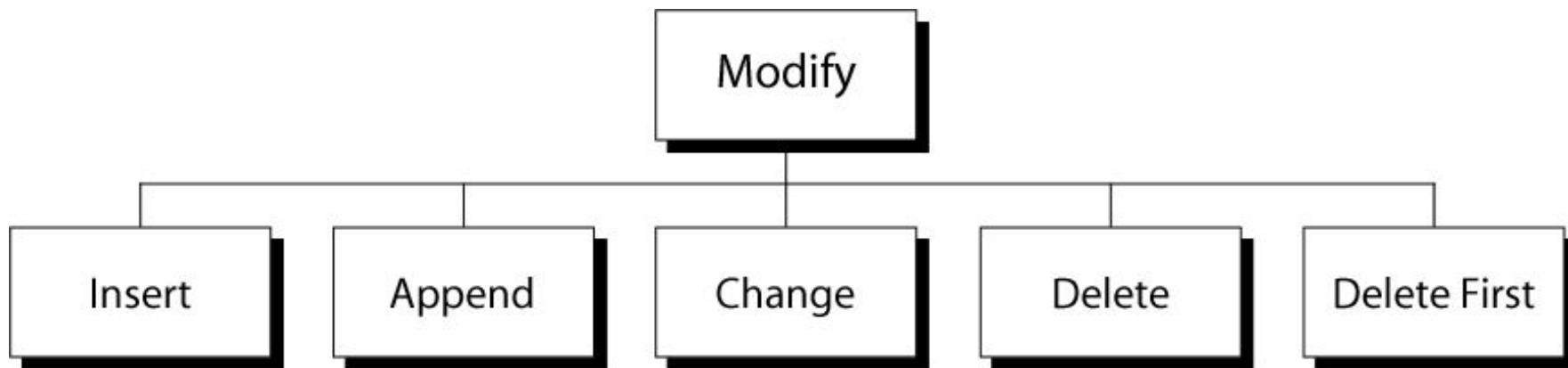
line command (=)

prints line numbers of matching statements

```
sed -n '20,30{/print/! =}' spy.py
```




Modify commands





Modify commands (contd.)

modify commands

- **insert**: sends the text immediately to the output. If the addressed line in pattern space is also sent to the output, it appears after the text.

```
sed '1 i 2019 Batch' Students.csv
```

inserts the text "2019 Batch" before the first line.

- **append**: leaves pattern space unchanged and queues text to be output after the current cycle ends.
- **change**: when the address matches, c removes the current line from pattern space, outputs the replacement text, and *moves to the next cycle*.

```
sed -n -r -e '/19D[0-9]{6}/ p' -e '/19D[0-9]{6}/ c Dual  
Degree Student' Students.csv
```

- **delete**: Delete the pattern space; immediately start next cycle.



Substitute command

Syntax: `s/search/replace/[flags]`

- Replaces the string that matched the search pattern with the replacement string.
search pattern can be a regular expression
- flags:
 - global (g), i.e. replace all occurrences
 - specific substitution count (integer), default 1
- Example: Remove all occurrences of the " character.
`sed -r 's/\\"//g' Students.csv`



Substitute command (contd.)

Substitution back references & and \n

- **&**: Refers to the text that matched the search pattern as &. The replacement pattern can refer to &

- s/ [red box] / [grey box] & [black box] /
- Convert roll nos to email addresses:

```
sed -r 's/19[0-9A-Z]{7}/&@iitb.ac.in/' Students.csv
```

- **\n** Refers to the text that matched the patterns in parentheses as \1, \2 etc. The replacement pattern can refer to these back references.

- s/ [grey box] ([red box]) [grey box] ([blue box]) [grey box] / [grey box] \1 [grey box] \2 [grey box] /

- Swap the roll no and the name columns:

```
sed -r 's/([19[0-9A-Z]{7})", "([a-zA-Z]*)"/\2", "\1/'  
Students.csv
```

12



Transform command y

- Syntax:

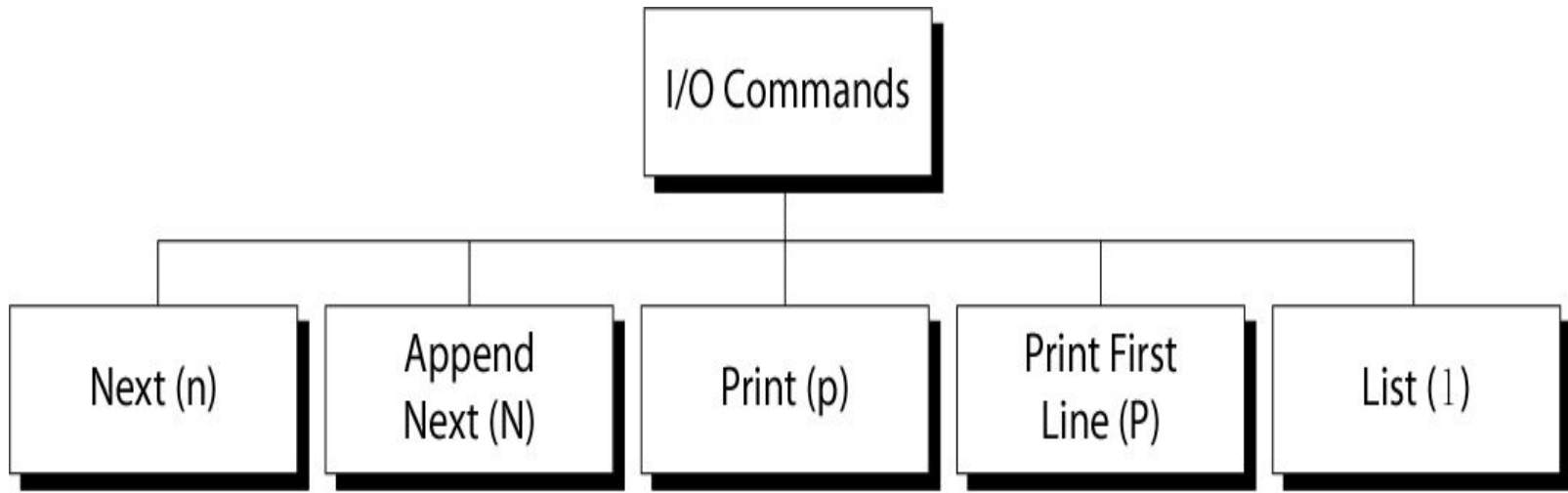
y/listofChars1/ListofChars2/

- Similar to the **tr** command. Replaces characters from the first list to corresponding characters of the second list.
- **listofChars1** and **listofChars2** should have the same length.
- Example:

```
sed '1,4y/BD/bd/' Students.csv
```



sed IO commands





sed IO commands n and N

(default sed workflow):

- finishes processing of the current line, optionally prints it
- reads the next input line into pattern space.
- resumes execution with the *next command in the script*.

```
sed 'n; n; a -----'
```

prints a dashed line after every third line.

● N

- After reading a line to the pattern space, add a newline, and then append the next line of input to the pattern space.
- Execute commands on the appended line.
- Print to output stream. Useful for processing to two or more lines at the same time.

```
sed -r 'N; s/\n//g' Students.csv
```

Joins adjacent lines removing the intervening newline.



sed IO commands p and P

- p
 - copies the entire contents of the pattern space to output
 - will print same line twice unless the option '-n' is used
- P
 - prints only the first line of the pattern space
 - prints the contents of the pattern space up to and including a new line character
 - any text following the first new line is not printed

```
sed -n 'N; p' Students.csv | wc -l
```

returns 146

```
sed -n 'N; P' Students.csv | wc -l
```

returns 73

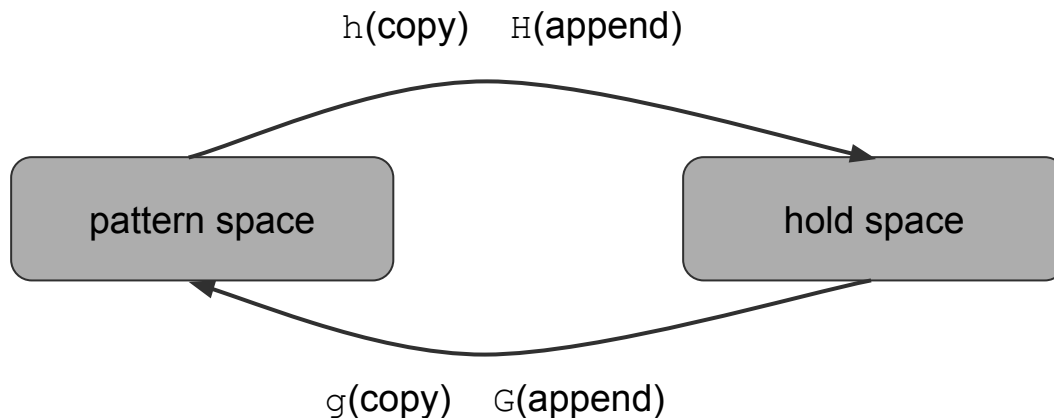


Moves between pattern and hold spaces

- To move text back and forth between pattern space and hold space.

`h : hold_space := pattern_space`
`H : hold_space := hold_space + "\n" + pattern_space`

`g : pattern_space := hold_space`
`G : pattern_space := pattern_space + "\n" + hold_space`



- The command `x` exchanges (swaps) the pattern and hold spaces
- Example: transfer the names of the dual degree students to the end.

```
sed -r -e '/19D[0-9]{6}/ H' -e '/19D[0-9]{6}/ d' -e '$ G' Students.csv
```



sed IO commands g,G,h and H.

- Same example in a file. To be run as `sed -r -f scriptfile Students.csv`

```
/19D[0-9]{6}/ {
```

```
    H
```

```
    d
```

```
}
```

```
$ {
```

```
    G
```

```
}
```

- The curly braces and the sed commands must appear on different lines.
- *The script has a flaw. Can you find and fix it?*



The sed workflow revised

```
hold_space := empty
while read_next_line(input, pattern_space) do
  append_queue := empty
  while there are any more commands cmd do
    if address_matches(cmd, pattern_space)
      case opcode(cmd) of
        's' : change pattern space by applying the substitution
        'p' : output(pattern_space)
        'i' : output(cmd.text)
        'a' : append_queue.enqueue(cmd.text)
        'd' : goto END_CYCLE
        'c' : output(cmd.text); goto END_CYCLE
        'n' : if auto_print then output(pattern_space);
              flush(append_queue);
              if not read_next_line(input, pattern_space) goto STOP
              else continue
        'N' : if not read_next_line(input, next_line) goto STOP
              else pattern_space := pattern_space + "\n" + next_line
      if auto_print then output(pattern_space);
    END_CYCLE: flush(append_queue)
```

19

STOP:



sed IO commands `r` and `w`.

- Allows reading from and writing to files.

- `r filename` – reads `filename` and writes it to the output stream. It is not copied to the pattern space.

```
sed '$r extras.csv' Students.csv
```

adds `extras.csv` to the output stream at the end of `Students.csv`

- `w filename` – writes from the pattern space to `filename`.

```
sed -r -n '/19D[0-9]{6}/w dd-students.csv' Students.csv
```

writes the matched students to the file `dd-students.csv`



sed IO commands b and q.

- b - branch unconditionally to label (or end of script)

```
/19D[0-9]{6}/ b save  
w others.csv  
b  
:save  
w dd-students.csv  
' Students.csv
```

Separates dual degree students and others into separate files.

- q - quits sed

```
sed '50q' datafile
```

Quits after printing the first 50 lines.



- Here is the fix for the flawed sed script.

```
/19D[0-9]{6}/{
```

```
$ {
```

```
    H
```

```
    x
```

```
    b
```

```
 }
```

```
 H
```

```
 d
```

```
}
```

```
$ {
```

```
    G
```

```
}
```

- if input line matches pattern
 - if input line is the last line
 - append line to hold space
 - hold swapped with pattern space
 - break out of script, auto_print prints all matched lines in pattern space
 - otherwise
 - transfer input line to hold
 - delete pattern space, end-of-cycle
- if input line does not match pattern
 - if input line is last
 - transfer hold to pattern space
 - auto_print prints last unmatched and all matched lines
 - otherwise
 - auto_print current line



awk

- Named after Aho, Weinberger and Kernighan
- Scripting language used for manipulating data and generating reports.
- The awk cycle:
 - scans a file line by line
 - splits each input line into fields
 - compares input line/fields to pattern
 - performs action(s) on matched lines
- Useful for:
 - transforming data files
 - producing formatted reports
- [This](#) is the main link for awk.



awk syntax

Basic awk syntax:

```
awk [options] 'script' file(s)  
awk [options] -f scriptfile file(s)
```

Options:

- F to change input field separator
- f to name script file



awk script

Consists of patterns & actions:

pattern {action}

- if pattern is missing, action is applied to all lines
- if action is missing, the matched line is printed
- must have either pattern or action

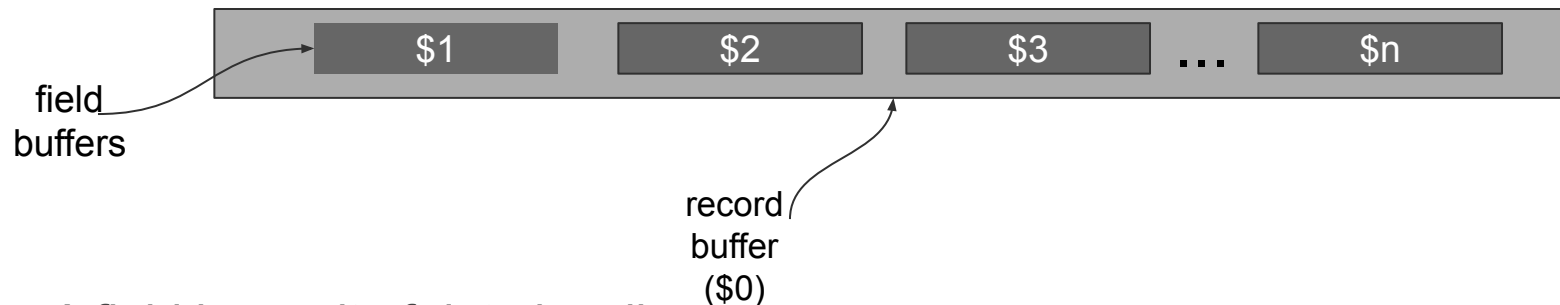
Example:

awk '/for/' testfile

- prints all lines containing string “for” in testfile



awk's view of data



- A field is a unit of data in a line
- Each field is separated from the other fields by a field separator
 - The default field separator is whitespace
- A record is the collection of fields in a line
- A data file is made up of records

Example of a record:

"190050339", "Kandibanda Sai", "ML", "Advisor : Prof. Dilip Jain"



awk predefined variables

- FS** -- Field separator (default=whitespace)
- RS** -- Record separator (default=\n)
- NF** -- Number of fields in current record
- NR** -- Number of the current record
- OFS** -- Output field separator (default=space)
- ORS** -- Output record separator (default=\n)
- FILENAME** -- Current filename

Example:

```
ls -al | awk '{print NR, $9}'
```

will number and print the files in the current directory.

```
awk -F, '/Aniket/{print NR, $1, $2}' Students.csv
```

print the line number, roll number and the full names of students named 'Aniket'₂₇



awk script structure

BEGIN {pre-processing statements}

Executed once before awk starts. For reading records, initializing variables, creating report headings

Pattern (action)

Pattern {action}

...

Pattern {action}

Executed once for each record in input file

END {post-processing statements}

Executed once after all records are read. For reporting aggregates (total, average), conclusion.



awk script structure

BEGIN: Pre-processing

- processing to be done before awk starts reading records from the input file
- useful for initialization: initialize variables, create report headings etc.

● Body

- contains logic to be applied to input file, one record at a time.

● END: Post-processing

- contains additional processing to be done on the records in the input file that have been processed
- Useful for reporting aggregates (total, average), conclusion.

● BEGIN and END are patterns

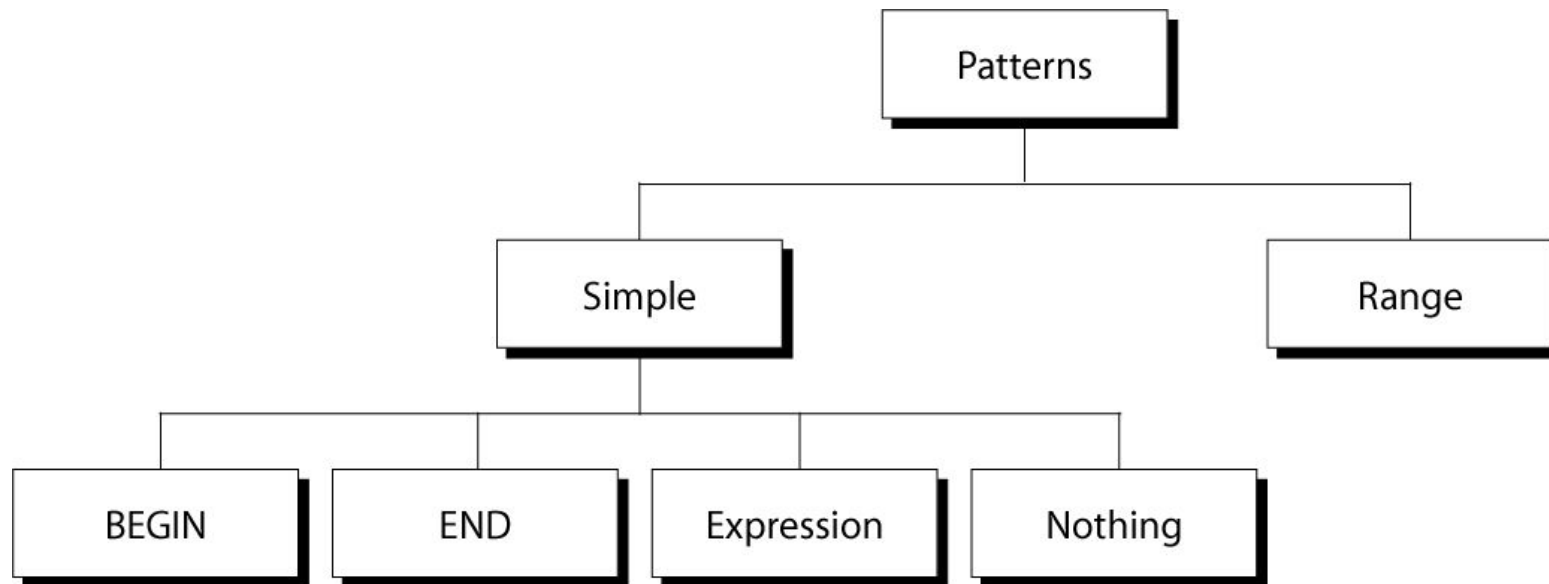


Structure of `awk` body

- `pattern {statement}`
- `pattern {statement; statement...statement}`
- `pattern {
 statement
 statement
 ...
 statement
}`



Categories of patterns





Pattern types: match expressions

Match expressions

- regular expression enclosed by '/'s (as in sed)
 - matches an occurrence in entire input record.
`awk -F, '/Pooja/ {print $1}' file`
Matches a record with the text **Pooja** anywhere
- explicit pattern-matching expressions ~ (match), !~ (not match)
 - Matches fields
`awk -F, '($1~/19D[A-Z0-9]+)/ {print NR, $0}' Students.csv`
print records whose first field match the regexp `19D[A-Z0-9]`
`+ /Pooja/{print}` is the same as `($0~/Pooja/)`

● ***Note: match means match anywhere within a record or a field***



Pattern types: non-match expressions

- Expressions made up of:
 - Constants: Numeric constants and strings.
 - Variables: Awk built-invariables, user defined variable, \$ variables.
 - Arithmetic operators: +, -, *, /, % (modulus), ^ (exponential)
 - String operators: Concatenation, search, substitution, extraction
 - Relational operators: <, <=, ==, !=, >, >=
 - Boolean operators: &&, ||, ! (not)

- Examples:

```
awk '$3 * $4 > 500 {print $0}' file
```

```
awk '($2 > 5) && ($2 <= 15){print $0}' file
```

```
awk '$3 == 100 || $4 > 50' file
```



Pattern type: range patterns

Matches ranges of consecutive input lines (much like sed)

- Syntax:

`pattern1, pattern2 {action}`

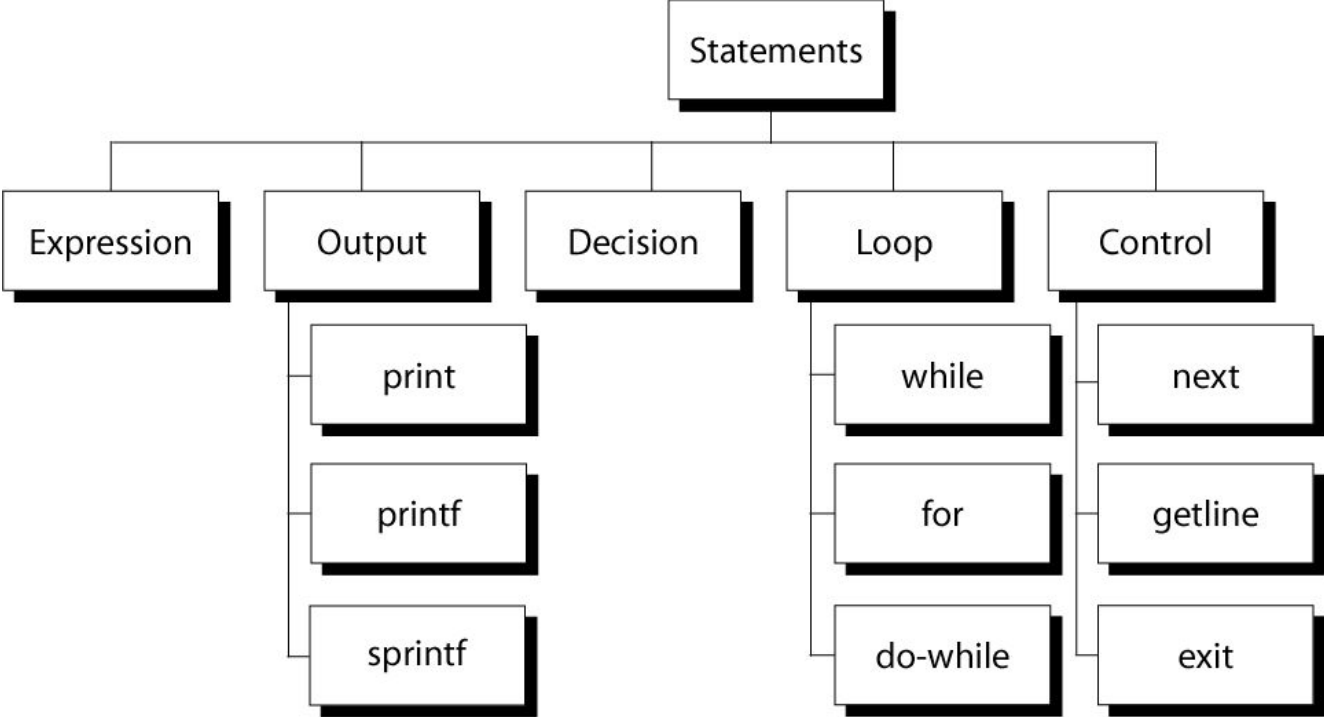
- pattern can be any simple pattern
- pattern1 turns action on
- pattern2 turns action off

- Example:

`awk /190050002/,/190020010/{print} Students.csv`



awk actions





awk actions

Made up of variables, constants and operators.

- a. Variables can be user variables, field variables and awk built-in variables.
- b. constants are numeric and string constants, associative arrays.

- Example: Feed the program input from `wc -c *`

`awk '`

`BEGIN {`

`lines=0; total=0; }`

`{`

`lines++; total+=$1;`

`}`

`END {`

`print lines " lines read"; print "total is ", total;`

`if (lines > 0 ) {`

`print "average is ", total/lines;`

`} else {`

`print "average is 0"; }}`



Variables and their scopes

- How to pass a shell variable to an awk script
 - a. `$1`, `$2` etc lose their shell variable identities. They become field variables.
 - b. The scope of a user defined variable does not extend to an awk script. The `-v` switch has to be used
 - c. The shaded part introduces an awk variable `w` in the shell, and passes to it the value of the shell `$w`. The variables need not have the same name.

```
printf "File: %s\n" "$1"
w=$1
awk -F, '
BEGIN {
    print "Printing w"; print w
}
/190050226/ {
    print $1; print "\n"
}' $1
```

```
printf "File: %s\n" "$1"
w=$1
awk -F, -v w="$w" '
BEGIN {
    print "Printing w"; print w
}
/190050226/ {
    print $1; print "\n"
}' $1
```



Associative arrays

- Does not have predefined indices. Indices get defined by use.

```
awk -F, '
($9~/[A-Z][A-Z]/){
    state[$9]=state[$9]+$10
}
END {
    for (i in state) {
        print state[i], i;
    }
}'
```

- \$9 is state code (TG, UP etc) and \$10 is the number of deaths. state is an array from state code to integers because of usage. A typical record is:

270226,,06/09/2020,,,,Medak,Telangana,TG,34,Hospitalized,,,https://t.me/indiacovid/16780,,,,,



Output statements

- **print**: print easy and simple output
- **printf**: print formatted (similar to C **printf**)
- **sprintf**: format string (similar to C **sprintf**)

Examples:

```
awk -F, '{print $1,$2 | "sort"}' Students.csv
```

```
awk -F, '{print $1,$2 | "sort -k 2"}' Students.csv
```

```
awk '{printf("z is %5.3f \n", z)}'
```

```
awk '{printf("The character is %c \n", x)}'
```

```
awk -F, '{ text = sprintf("1: %d - 2: %d", $1, $2); print  
text}' Students.csv
```



Decision and loops

- if statements
- for statement

```
for (i = 1; i <= NR; i++)
```

- {
 total += \$i
 count++
}

- while-do,do-while
- break,continue

```
for (x = 0; x < 20; x++) {  
    if ( array[x] > 100) continue  
    printf "%d ", x  
    if ( array[x] < 0 ) break  
}
```




Example

```
BEGIN {
```

```
FS= ","; total = 0
```

```
print "      Covid Data      Date: 24th July"
```

```
print "      State      Dist.                  Cases"
```

```
print "      =====
```

```
printf("%6s %-25s  %d\n", $9, $7, $10) | "sort -k 1"
```

```
state[$9] += $10; total += $10
```

```
END {
```

```
print "      =====
```

```
print "      State                  Total"
```

```
print "      =====
```

```
for (i in state)
```

```
{
```

```
    printf("      %-15s                  %d\n", i, state[i])
```

```
}
```

```
print "      =====
```

```
print "      Total Cases (India):                  " total}
```